

1.3 MUST KNOW DETAILS

MIKROELEKTRONIKA

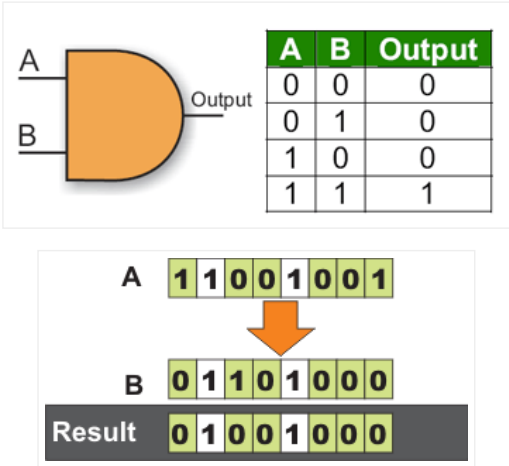
The operation of these elements is based on principles established by a British mathematician *George Boole* in the middle of the 19th century- even before the first bulb was invented. Originally, the main idea was to express logical forms through algebraic functions. Such thinking was soon transformed into a practical product which far later evaluated in what today is known as AND, OR and NOT logic circuits. The principle of their operation is known as Boolean algebra.

LOGIC CIRCUITS

Some of the program instructions give the same results as logic gates. The principle of their operation will be discussed in the text below.

AND Gate

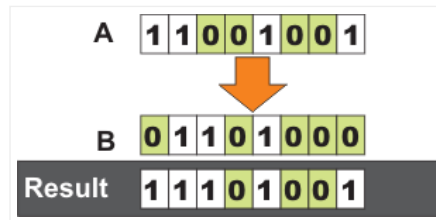
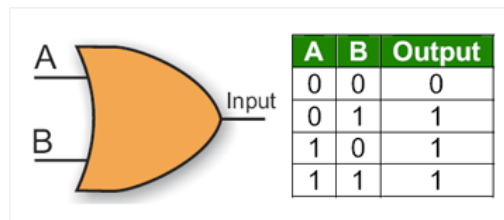
The logic gate ‘AND’ has two or more inputs and one output. Let us presume that the gate used in this example has only two inputs. A logic one (1) will appear on its output only if both inputs (A AND B) are driven high (1). Table on the right shows mutual dependence between inputs and the output.



When used in a program, a logic AND operation is performed by the program instruction, which will be discussed later. For the time being, it is enough to remember that logic AND in a program refers to the corresponding bits of two registers.

OR GATE

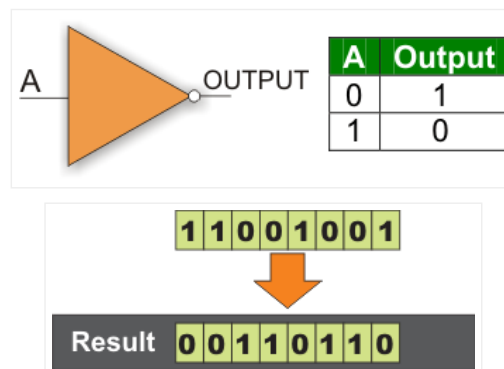
Similarly, OR gates also have two or more inputs and one output. If the gate has only two inputs the following applies. A logic one (1) will appear on its output if either input (A OR B) is driven high (1). If the OR gate has more than two inputs then the following applies. A logic one (1) appears on its output if at least one input is driven high (1). If all inputs are at logic zero (0), the output will be at logic zero (0) as well.



In the program, logic OR operation is performed in the same manner as logic AND operation.

NOT GATE

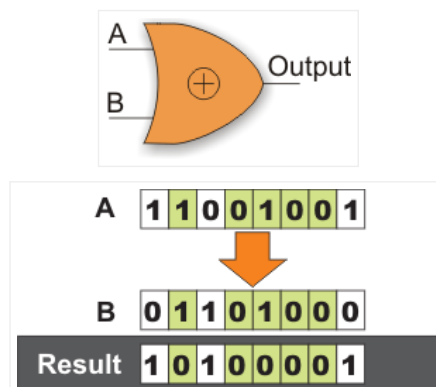
The logic gate NOT has only one input and only one output. It operates in an extremely simple way. When logic zero (0) appears on its input, a logic one (1) appears on its output and vice versa. It means that this gate inverts the signal and is often called inverter, therefore.



In the program, logic NOT operation is performed upon one byte. The result is a byte with inverted bits. If byte bits are considered to be a number, the inverted value is actually a complement thereof. The complement of a number is a value which added to the number makes it reach the largest 8-digit binary number. In other words, the sum of an 8-digit number and its complement is always 255.

EXCLUSIVE OR GATE

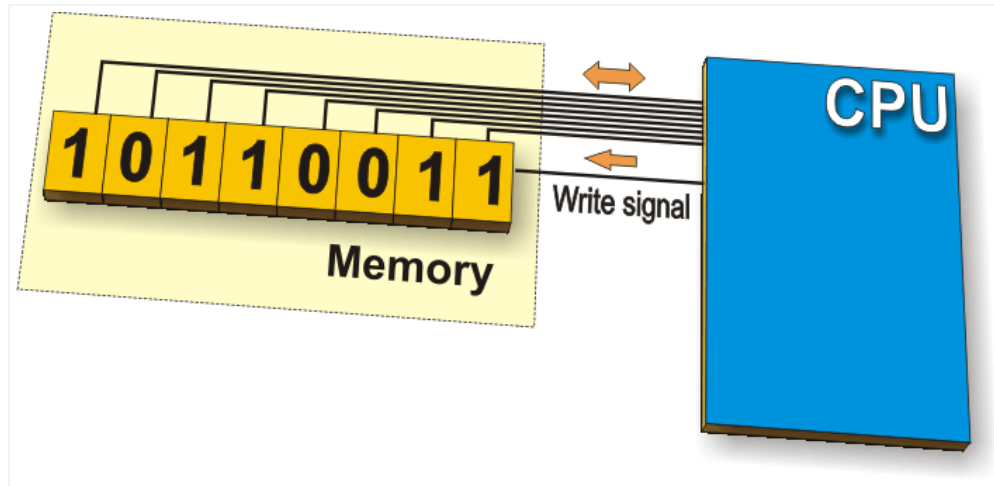
The EXCLUSIVE OR (XOR) gate is a bit complicated comparing to other gates. It represents a combination of all of them. A logic one (1) appears on its output only when its inputs have different logic states.



In the program, this operation is commonly used to compare two bytes. Subtraction may be used for the same purpose (if the result is 0, bytes are equal). Unlike subtraction, the advantage of this logic operation is that it is not possible to obtain negative results.

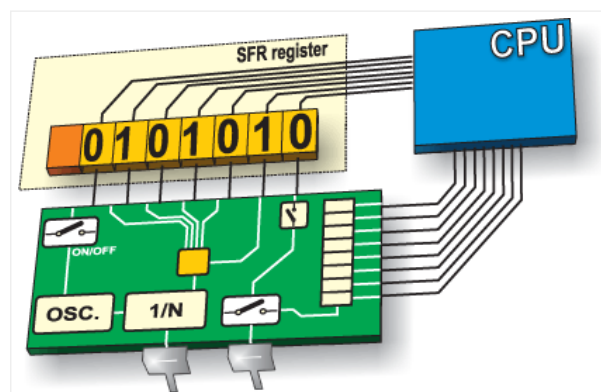
REGISTER

In short, a register or a memory cell is an electronic circuit which can memorize the state of one byte.



SFR REGISTERS

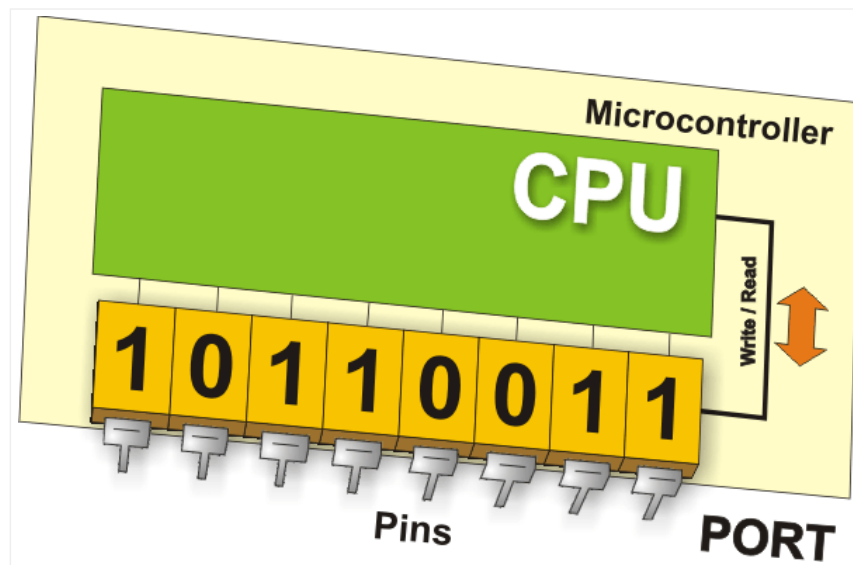
In addition to registers which do not have any special and predetermined function, every microcontroller has a number of registers (SFR) whose function is predetermined by the manufacturer. Their bits are connected (literally) to internal circuits of the microcontroller such as timers, A/D converter, oscillators and others, which means that they are directly in command of the operation of these circuits, i.e. the microcontroller. Imagine eight switches which control the operation of a small circuit within the microcontroller- Special Function Registers do exactly that.



In other words, the state of register bits is changed from within the program, registers run small circuits within the microcontroller, these circuits are via microcontroller pins connected to peripheral electronics which is used for... Well, it's up to you.

INPUT / OUTPUT PORTS

In order to make the microcontroller useful, it has to be connected to additional electronics, i.e. peripherals. Each microcontroller has one or more registers (called ports) connected to the microcontroller pins. Why input/output? Because you can change a pin function as you wish. For example, suppose you want your device to turn on/off three signal LEDs and simultaneously monitor the logic state of five sensors or push buttons. Some of the ports need to be configured so that there are three outputs (connected to LEDs) and five inputs (connected to sensors). It is simply performed by software, which means that a pin function can be changed during operation.



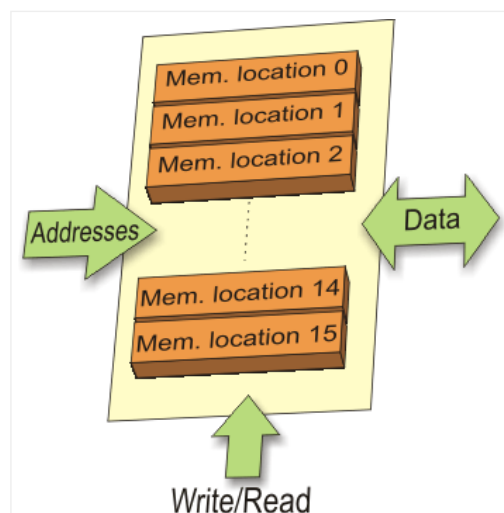
One of important specifications of input/output (I/O) pins is the maximum current they can handle. For most microcontrollers, current obtained from one pin is sufficient to activate an LED or some other low-current device (10-20 mA). The more I/O pins, the lower maximum current of one pin. In other words, the maximum current stated in the data specifications sheet for the microprocessor is shared across all I/O ports.

Another important pin function is that it can have pull-up resistors. These resistors connect pins to the positive power supply voltage and come into effect when the pin is configured as an input connected to a mechanical switch or a push button. Newer versions of microcontrollers have pull-up resistors configurable by software.

Each I/O port is usually under control of the specialized SFR, which means that each bit of that register determines the state of the corresponding microcontroller pin. For example, by writing logic one (1) to a bit of the control register (SFR), the appropriate port pin is automatically configured as an input and voltage brought to it can be read as logic 0 or 1. Otherwise, by writing zero to the SFR, the appropriate port pin is configured as an output. Its voltage (0V or 5V) corresponds to the state of appropriate port register bit.

MEMORY UNIT

Memory is part of the microcontroller used for data storage. The easiest way to explain it is to compare it with a filing cabinet with many drawers. Suppose, the drawers are clearly marked so that their contents can be easily found out by reading the label on the front of the drawer.



Similarly, each memory address corresponds to one memory location. The contents of any location can be accessed and read by its addressing. Memory can either be written to or read from. There are several types of memory within the microcontroller:

READ ONLY MEMORY (ROM)

Read Only Memory (ROM) is used to permanently save the program being executed. The size of program that can be written depends on the size of this memory. Today's microcontrollers commonly use 16-bit addressing, which means that they are able to address up to 64 Kb of memory, i.e. 65535 locations. As a novice, your program will rarely exceed the limit of several hundred instructions. There are several types of ROM.

Masked ROM (MROM)

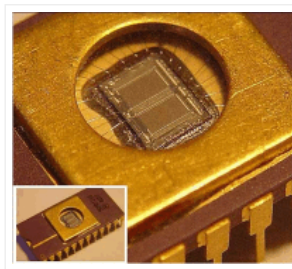
Masked ROM is a kind of ROM the content of which is programmed by the manufacturer. The term 'masked' comes from the manufacturing process, where regions of the chip are masked off before the process of photolithography. In case of a large-scale production, the price is very low. Forget it...

One Time Programmable ROM (OTP ROM)

One time programmable ROM enables you to download a program into it, but, as its name states, one time only. If an error is detected after downloading, the only thing you can do is to download the correct program to another chip.

UV Erasable Programmable ROM (UV EPROM)

Installation of this window is complicated, which normally affects the price. From our point of view, unfortunately-negative...



Both the manufacturing process and characteristics of this memory are completely identical to OTP ROM. However, the package of the microcontroller with this memory has a recognizable 'window' on its top side. It enables data to be erased under strong ultraviolet light. After a few minutes it is possible to download a new program into it.

Flash Memory

This type of memory was invented in the 80s in the laboratories of INTEL and was represented as the successor to the UV EPROM. Since the content of this memory can be written and cleared practically an unlimited number of times, microcontrollers with Flash ROM are ideal for learning, experimentation and small-scale production. Because of its great popularity, most microcontrollers are manufactured in flash technology today. So, if you are going to buy a microcontroller, the type to look for is definitely Flash!

RANDOM ACCESS MEMORY (RAM)

Once the power supply is off the contents of RAM is cleared. It is used for temporary storing data and intermediate results created and used during the operation of the microcontroller. For example, if the program performs an addition (of whatever), it is necessary to have a register representing what in everyday life is called the 'sum'. For this reason, one of the registers of RAM is called the 'sum' and used for storing results of addition.

ELECTRICALLY ERASABLE PROGRAMMABLE ROM (EEPROM)

The contents of EEPROM may be changed during operation (similar to RAM), but remains permanently saved even after the loss of power (similar to ROM). Accordingly, EEPROM is often used to store values, created during

operation, which must be permanently saved. For example, if you design an electronic lock or an alarm, it would be great to enable the user to create and enter the password, but it's useless if lost every time the power supply goes off. The ideal solution is a microcontroller with an embedded EEPROM.

INTERRUPT

Most programs use interrupts in their regular execution. The purpose of the microcontroller is mainly to respond to changes in its surrounding. In other words, when an event takes place, the microcontroller does something... For example, when you push a button on a remote controller, the microcontroller will register it and respond by changing a channel, turn the volume up or down etc. If the microcontroller spent most of its time endlessly checking a few buttons for hours or days, it would not be practical at all.

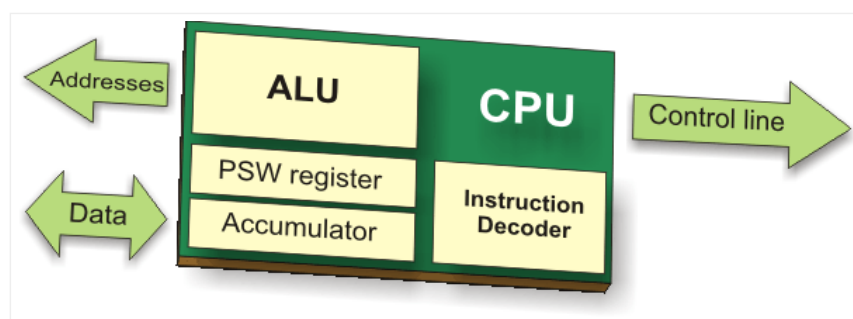
This is why the microcontroller has learnt a trick during its evolution. Instead of checking each pin or bit constantly, the microcontroller delegates the 'wait issue' to a 'specialist' which will respond only when something attention worthy happens.

The signal which informs the central processor unit about such an event is called an INTERRUPT.

CENTRAL PROCESSOR UNIT (CPU)

As its name suggests, this is a unit which monitors and controls all processes within the microcontroller. It consists of several subunits, of which the most important are:

- **Instruction Decoder** is a part of electronics which decodes program instructions and runs other circuits on the basis of that. The 'instruction set' which is different for each microcontroller family expresses the abilities of this circuit;
- **Arithmetical Logical Unit (ALU)** performs all mathematical and logical operations upon data; and
- **Accumulator** is an SFR closely related to the operation of the ALU. It is a kind of working desk used for storing all data upon which some operation should be performed (addition, shift/move etc.). It also stores results ready for use in further processing. One of the SFRs, called a *Status Register (PSW)*, is closely related to the accumulator. It shows at any given time the 'status' of a number stored in the accumulator (number is larger or less than zero etc.). Accumulator is also called working register and is marked as W register or just W, therefore.



BUS

A bus consists of 8, 16 or more wires. There are two types of buses: the address bus and the data bus. The address bus consists of as many lines as necessary for memory addressing. It is used to transmit address from the CPU to the memory. The data bus is as wide as the data, in our case it is 8 bits or wires wide. It is used to connect all the circuits within the microcontroller.

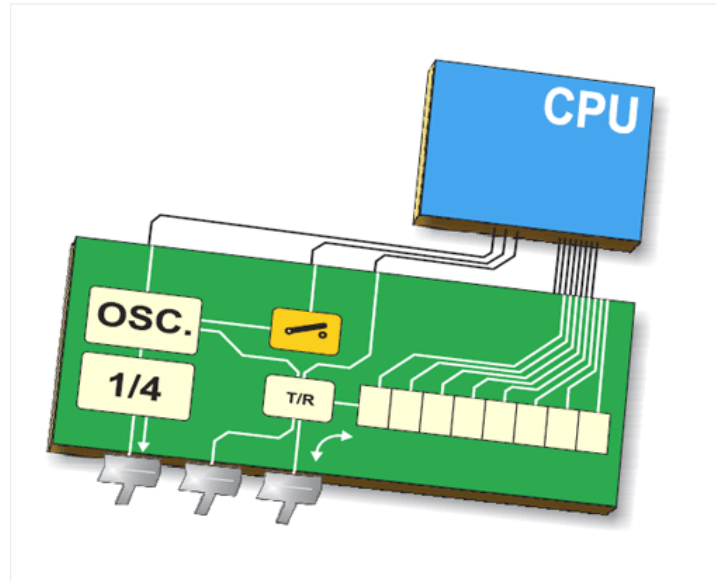
SERIAL COMMUNICATION

Parallel connection between the microcontroller and peripherals via input/output ports is the ideal solution on shorter distances up to several meters. However, in other cases when it is necessary to establish communication

between two devices on longer distances it is not possible to use parallel connection. Instead, serial communication is used.

Today, most microcontrollers have built in several different systems for serial communication as a standard equipment. Which of these systems will be used depends on many factors of which the most important are:

- How many devices the microcontroller has to exchange data with?
- How fast the data exchange has to be?
- What is the distance between devices?
- Is it necessary to send and receive data simultaneously?



One of the most important things concerning serial communication is the *Protocol* which should be strictly observed. It is a set of rules which must be applied in order that devices can correctly interpret data they mutually exchange. Fortunately, the microcontroller automatically takes care of this, so that the work of the programmer/user is reduced to simple write (data to be sent) and read (received data).

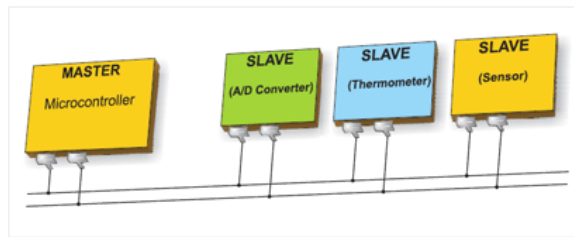
BAUD RATE

The term *baud rate* is used to denote the number of bits transferred per second [bps]. Note that it refers to bits, not bytes. It is usually required by the protocol that each byte is transferred along with several control bits. It means that one byte in serial data stream may consist of 11 bits. For example, if the baud rate is 300 bps then maximum 37 and minimum 27 bytes may be transferred per second.

The most commonly used serial communication systems are:

I²C (INTER INTEGRATED CIRCUIT)

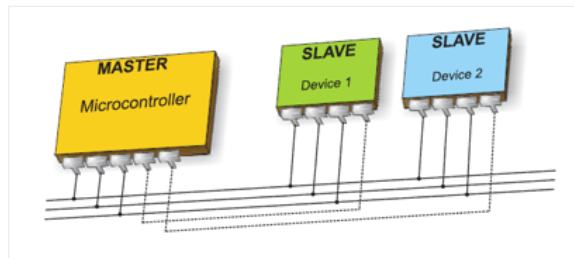
Inter-integrated circuit is a system for serial data exchange between the microcontrollers and specialized integrated circuits of a new generation. It is used when the distance between them is short (receiver and transmitter are usually on the same printed board). Connection is established via two conductors. One is used for data transfer, the other is used for synchronization (clock signal). As seen in figure below, one device is always a master. It performs addressing of one slave chip before communication starts. In this way one microcontroller can communicate with 112 different devices. Baud rate is usually 100 Kb/sec (standard mode) or 10 Kb/sec (slow baud rate mode). Systems with the baud rate of 3.4 Mb/sec have recently appeared. The distance between devices which communicate over an I²C bus is limited to several meters.



SPI (SERIAL PERIPHERAL INTERFACE BUS)

A serial peripheral interface (SPI) bus is a system for serial communication which uses up to four conductors, commonly three. One conductor is used for data receiving, one for data sending, one for synchronization and one alternatively for selecting a device to communicate with. It is a full duplex connection, which means that data is sent and received simultaneously.

The maximum baud rate is higher than that in the I2C communication system.

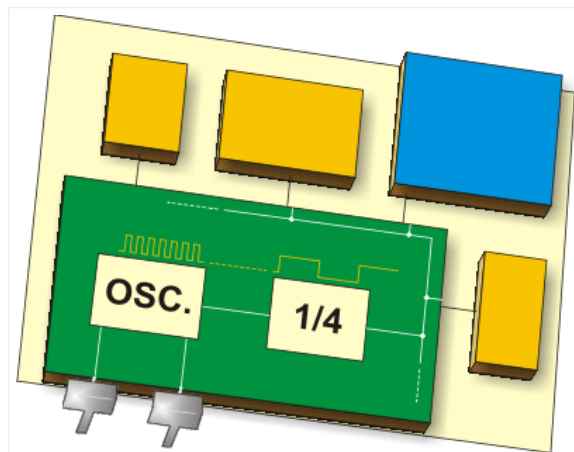


UART (UNIVERSAL ASYNCHRONOUS RECEIVER/TRANSMITTER)

This sort of communication is asynchronous, which means that a special line for transferring clock signal is not used. In some applications, such as radio connection or infrared waves remote control, this feature is crucial. Since only one communication line is used, both receiver and transmitter operate at the same predefined rate in order to maintain necessary synchronization. This is a very simple way of transferring data since it basically represents the conversion of 8-bit data from parallel to serial format. Baud rate is not high, up to 1 Mbit/sec.

OSCILLATOR

Even pulses generated by the oscillator enable harmonic and synchronous operation of all circuits within the microcontroller. The oscillator is usually configured so as to use quartz crystal or ceramic resonator for frequency stability, but it can also operate as a stand-alone circuit (like RC oscillator). It is important to say that instructions are not executed at the rate imposed by the oscillator itself, but several times slower. It happens because each instruction is executed in several steps. In some microcontrollers, the same number of cycles is needed to execute all instructions, while in others, the number of cycles is different for different instructions. Accordingly, if the system uses quartz crystal with a frequency of 20 Mhz, the execution time of an instruction is not 50nS, but 200, 400 or 800 nS, depending on the type of MCU!



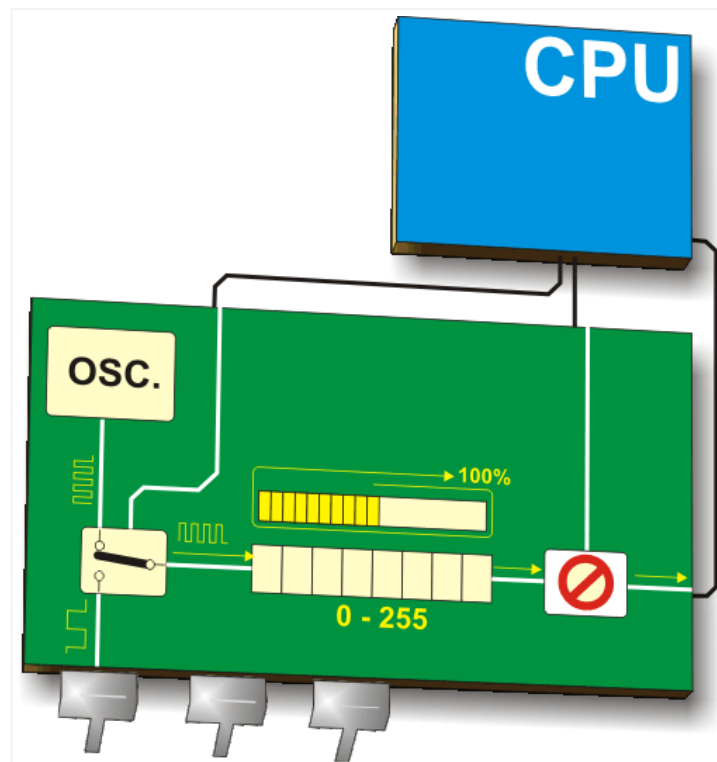
POWER SUPPLY CIRCUIT

There are two things worth attention concerning the microcontroller power supply circuit:

- **Brown out** is a potentially dangerous condition which occurs at the moment the microcontroller is being turned off or when the power supply voltage drops to a minimum due to electric noise. As the microcontroller consists of several circuits with different operating voltage levels, this state can cause its out-of-control performance. In order to prevent it, the microcontroller usually has a built-in circuit for brown out reset which resets the whole electronics as soon as the microcontroller incurs a state of emergency.
- **Reset pin** is usually marked as MCLR (*Master Clear Reset*). It is used for external reset of the microcontroller by applying a logic zero (0) or one (1) to it, which depends on the type of the microcontroller. In case the brown out circuit is not built in, a simple external circuit for brown out reset can be connected to the MCLR pin.

TIMERS/COUNTERS

The microcontroller oscillator uses quartz crystal for its operation. Even though it is not the simplest solution, there are many reasons to use it. The frequency of such oscillator is precisely defined and very stable, so that pulses it generates are always of the same width, which makes them ideal for time measurement. Such oscillators are also used in quartz watches. If it is necessary to measure time between two events, it is sufficient to count up pulses generated by this oscillator. This is exactly what the timer does.



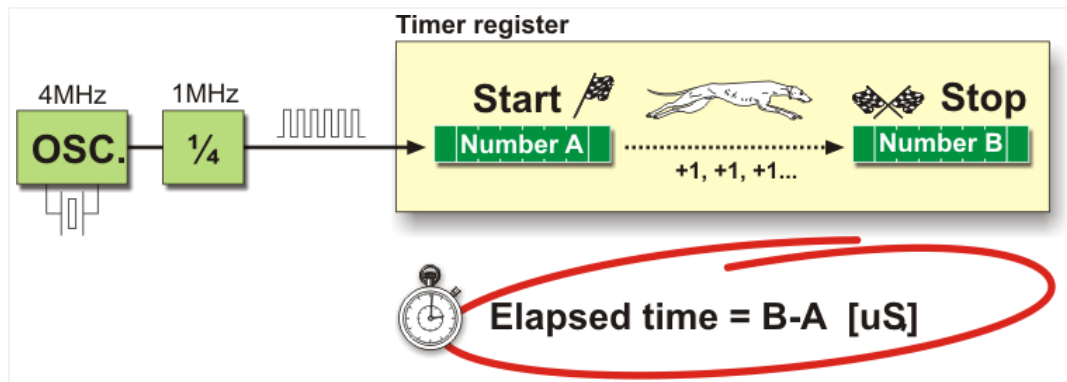
Most programs use these miniature electronic ‘stopwatches’. These are commonly 8- or 16-bit SFRs the contents of which is automatically incremented by each coming pulse. Once a register is completely loaded, an interrupt may be generated!

If the timer uses an internal quartz oscillator for its operation then it can be used to measure time between two events (if the register value is T1 at the moment measurement starts, and T2 at the moment it terminates, then the elapsed time is equal to the result of subtraction T2-T1). If registers use pulses coming from external source then such a timer is turned into a counter.

This is only a simple explanation of the operation itself. It is however more complicated in practice.

HOW DOES THE TIMER OPERATE?

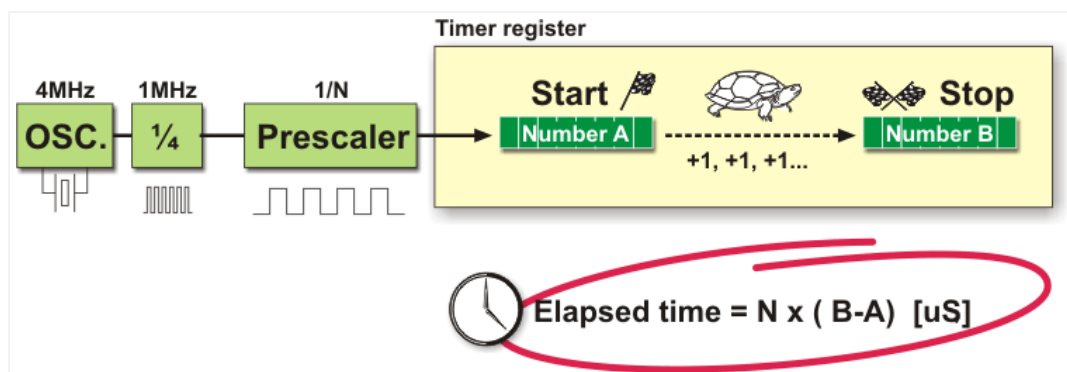
In practice, pulses generated by the quartz oscillator are once per each machine cycle, directly or via a prescaler, brought to the circuit which increments the number stored in the timer register. If one instruction (one machine cycle) lasts for four quartz oscillator periods then this number will be incremented a million times per second (each microsecond) by embedding quartz with the frequency of 4MHz.



It is easy to measure short time intervals, up to 256 microseconds, in the way described above because it is the largest number that one register can store. This restriction may be easily overcome in several ways such as by using a slower oscillator, registers with more bits, prescaler or interrupts. The first two solutions have some weaknesses so it is more recommended to use prescalers or interrupts.

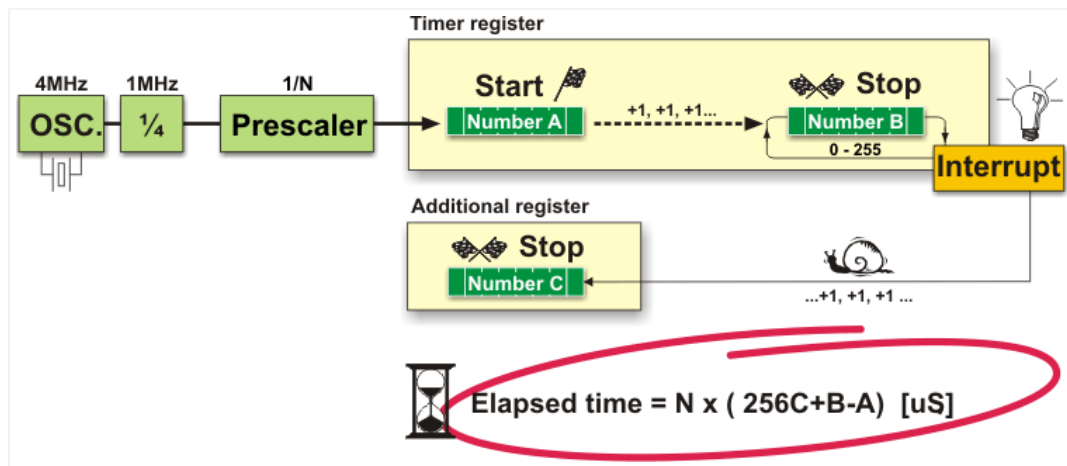
USING A PRESCALER IN TIMER OPERATION

A prescaler is an electronic device used to reduce frequency by a predetermined factor. In order to generate one pulse on its output, it is necessary to bring 1, 2, 4 or more pulses on its input. Most microcontrollers have one or more prescalers built in and their division rate may be changed from within the program. The prescaler is used when it is necessary to measure longer periods of time. If one prescaler is shared by timer and watchdog timer, it cannot be used by both of them simultaneously.



USING INTERRUPT IN TIMER OPERATION

If the timer register consists of 8 bits, the largest number it can store is 255. As for 16-bit registers it is the number 65,535. If this number is exceeded, the timer will be automatically reset and counting will start at zero again. This condition is called an overflow. If enabled from within the program, the overflow can cause an interrupt, which gives completely new possibilities. For example, the state of registers used for counting seconds, minutes or days can be changed in an interrupt routine. The whole process (except for interrupt routine) is automatically performed behind the scenes, which enables the main circuits of the microcontroller to operate normally.



This figure illustrates the use of an interrupt in timer operation. Delays of arbitrary duration, having almost no influence on the main program execution, can be easily obtained by assigning the prescaler to the timer.

COUNTERS

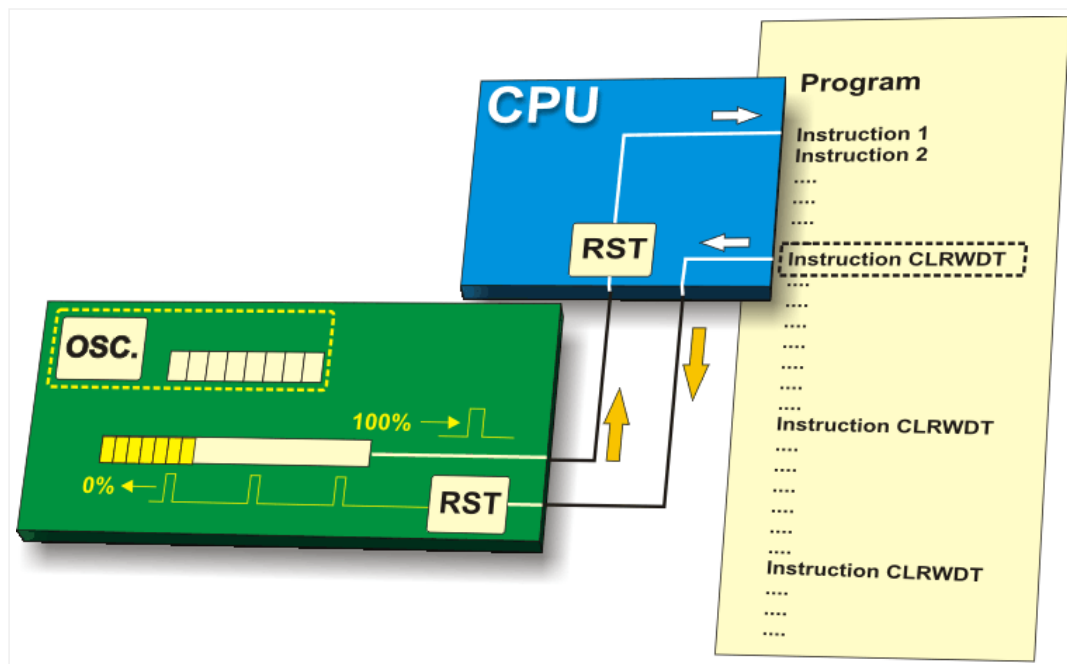
If the timer receives pulses from the microcontroller input pin, then it turns into a counter. Obviously, it is the same electronic circuit able to operate in two different modes. The only difference is that in this case pulses to be counted come over the microcontroller input pin and their duration (width) is mostly undefined. This is why they cannot be used for time measurement, but for other purposes such as counting products on an assembly line, number of axis rotation, passengers etc. (depending on sensor in use).

WATCHDOG TIMER

A watchdog timer is a timer connected to a completely separate RC oscillator within the microcontroller.

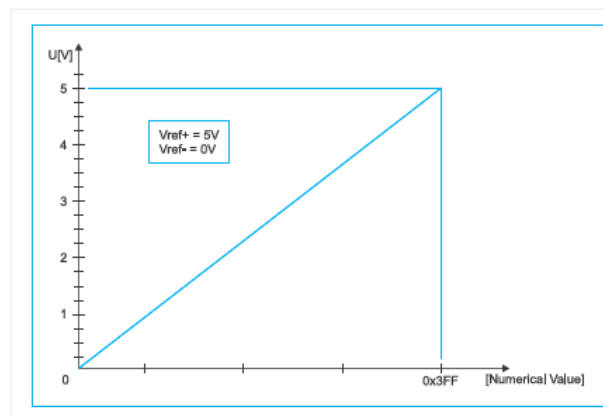
If the watchdog timer is enabled, every time it counts up to the maximum value, the microcontroller reset occurs and the program execution starts from the first instruction. The point is to prevent this from happening by using a specific command.

Anyway, the whole idea is based on the fact that every program is executed in several longer or shorter loops. If instructions which reset the watchdog timer are set at the appropriate program locations, besides commands being regularly executed, then the operation of the watchdog timer will not affect the program execution. If for any reason, usually electrical noise in industry, the program counter 'gets stuck' at some memory location from which there is no return, the watchdog timer will not be cleared, so the register's value being constantly incremented will reach the maximum *et voila!* Reset occurs!

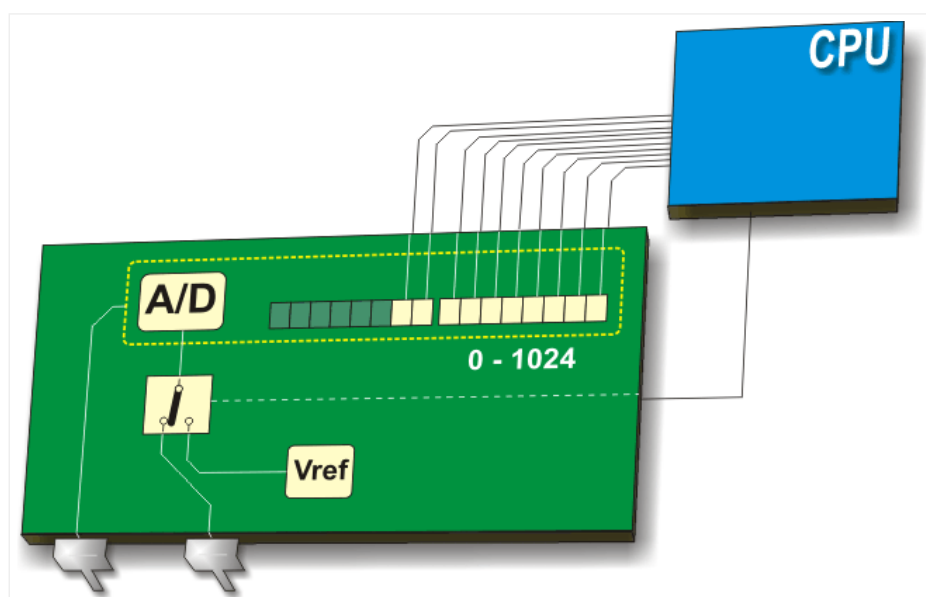


A/D CONVERTER

External signals are usually fundamentally different from those the microcontroller understands (ones and zeros) and have to be converted therefore into values understandable for the microcontroller. An analogue to digital converter is an electronic circuit which converts continuous signals to discrete digital numbers. In other words, this circuit converts an analogue value into a binary number and passes it to the CPU for further processing. This module is therefore used for input pin voltage measurement (analogue value).



The result of measurement is a number (digital value) used and processed later in the program.



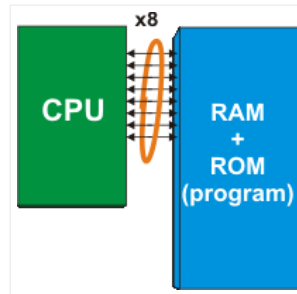
INTERNAL ARCHITECTURE

All upgraded microcontrollers use one of two basic design models called *Harvard* and *von-Neumann* architecture.

They represent two different ways of exchanging data between CPU and memory.

VON-NEUMANN ARCHITECTURE

Microcontrollers using von-Neumann architecture have only one memory block and one 8-bit data bus. As all data are exchanged through these 8 lines, the bus is overloaded and communication is very slow and inefficient. The CPU can either read an instruction or read/write data from/to the memory. Both cannot occur at the same time since instructions and data use the same bus. For example, if a program line reads that RAM memory register called 'SUM' should be incremented by one (instruction: `incf SUM`), the microcontroller will do the following:

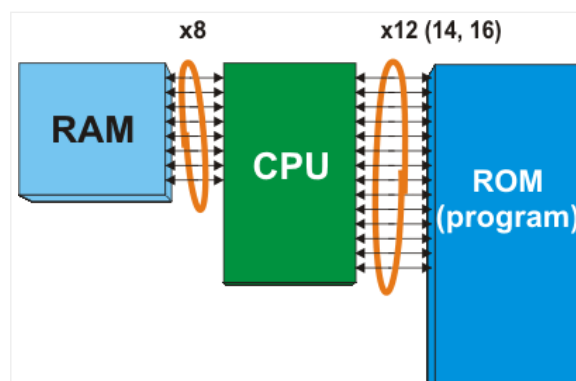


1. Read the part of the program instruction specifying WHAT should be done (in this case it is the 'incf' instruction for increment).
2. Read the other part of the same instruction specifying upon WHICH data it should be performed (in this case it is the 'SUM' register).
3. After being incremented, the contents of this register should be written to the register from which it was read ('SUM' register address).

The same data bus is used for all these intermediate operations.

HARVARD ARCHITECTURE

Microcontrollers using Harvard architecture have two different data buses. One is 8 bits wide and connects CPU to RAM. The other consists of 12, 14 or 16 lines and connects CPU to ROM. Accordingly, the CPU can read an instruction and access data memory at the same time. Since all RAM memory registers are 8 bits wide, all data being exchanged are of the same width. During the process of writing a program, only 8-bit data are considered. In other words, all you can change from within the program and all you can influence is 8 bits wide. All the programs written for these microcontrollers will be stored in the microcontroller internal ROM after being compiled into machine code. However, ROM memory locations do not have 8, but 12, 14 or 16 bits. The rest of bits 4, 6 or 8 represents instruction specifying for the CPU what to do with the 8-bit data.



The advantages of such design are the following:

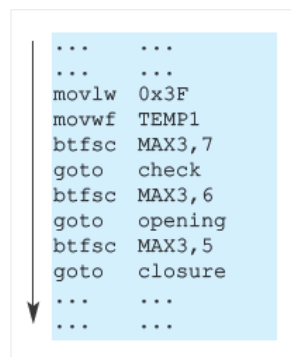
- All data in the program is one byte (8 bits) wide. As the data bus used for program reading has 12, 14 or 16

lines, both instruction and data can be read simultaneously using these spare bits. For this reason, all instructions are single-cycle instructions, except for the jump instruction which is two-cycle instruction.

- Owing to the fact that the program (ROM) and temporary data (RAM) are separate, the CPU can execute two instructions at a time. Simply put, while RAM read or write is in progress (the end of one instruction), the next program instruction is read through the other bus.
- When using microcontrollers with von-Neumann architecture, one never knows how much memory is to be occupied by the program. Basically, most program instructions occupy two memory locations (one contains information on WHAT should be done, whereas the other contains information upon WHICH data it should be done). However, it is not a hard and fast rule, but the most common case. In microcontrollers with Harvard architecture, the program word bus is wider than one byte, which allows each program word to consist of instruction and data, i.e. one memory location – one program instruction.

INSTRUCTION SET

All instructions understandable to the microcontroller are called together the *Instruction Set*. When you write a program in assembly language, you actually specify instructions in such an order they should be executed. The main restriction here is a number of available instructions. The manufacturers usually adopt either approach described below:



RISC (REDUCED INSTRUCTION SET COMPUTER)

In this case, the microcontroller recognizes and executes only basic operations (addition, subtraction, copying etc.). Other, more complicated operations are performed by combining them. For example, multiplication is performed by performing successive addition. It's the same as if you try to explain to someone, using only a few different words, how to reach the airport in a new city. However, it's not as black as it's painted. First of all, this language is easy to learn. The microcontroller is very fast so that it is not possible to see all the arithmetic 'acrobatics' it performs. The user can only see the final results. At last, it is not so difficult to explain where the airport is if you use the right words such as left, right, kilometers etc.

CISC (COMPLEX INSTRUCTION SET COMPUTER)

CISC is the opposite to RISC! Microcontrollers designed to recognize more than 200 different instructions can do a lot of things at high speed. However, one needs to understand how to take all that such a rich language offers, which is not at all easy...

HOW TO MAKE THE RIGHT CHOICE?

Ok, you are the beginner and you have made a decision to go on an adventure of working with the microcontrollers. Congratulations on your choice! However, it is not as easy to choose the right microcontroller as it may seem. The problem is not a limited range of devices, but the opposite!

Before you start to design a device based on the microcontroller, think of the following: how many input/output lines will I need for operation? Should it perform some other operations than to simply turn relays on/off? Does it need some specialized module such as serial communication, A/D converter etc.? When you create a clear

picture of what you need, the selection range is considerably reduced and it's time to think of price. Are you planning to have several same devices? Several hundred? A million? Anyway, you get the point.

If you think of all these things for the very first time then everything seems a bit confusing. For this reason, go step by step. First of all, select the manufacturer, i.e. the microcontroller family you can easily get. Study one particular model. Learn as much as you need, don't go into details. Solve a specific problem and something incredible will happen- you will be able to handle any model belonging to that microcontroller family.

Remember learning to ride a bicycle. After several bruises at the beginning, you were able to keep balance, then to easily ride any other bicycle. And of course, you will never forget programming just as you will never forget riding bicycles!



1.3 Must know details by MikroElektronika is licensed under a Creative Commons Attribution 4.0 International License, except where otherwise noted.